A quick introduction to LDscnR

Contents

0.1 Overview . . . 1
0.2 Example data . . . 1
0.3 Estimating LD decay . . . 2
0.4 Repeated outlier-region draws . . . 6
0.5 Manhattan-style visualization . . . 7
0.6 Benchmarking with simulated truth . . . 8
0.7 Area under the PR curve . . . 11
0.8 Summary . . . 12


```r
library(LDscnR)
library(data.table)
library(SNPRelate)
library(ggplot2)
library(patchwork)
```

0.1 Overview

`LDscnR` provides tools for summarizing genome scan results in the presence of linkage disequilibrium (LD). The package estimates chromosome-specific LD decay, derives LD-aware scan statistics and outlier regions, and summarizes robustness across parameter settings using consistency scores.

This vignette gives a compact end-to-end example using the bundled simulated dataset `sim_ex`. The workflow covers:

- creation of a GDS object from genotype data,
- estimation of LD decay,
- generation of repeated outlier-region draws,
- visualization of consistency scores and outlier regions,
- evaluation of performance in simulated data using precision-recall summaries.

0.2 Example data

The example dataset contains:

- `GTs`: a genotype matrix,
- `map`: a SNP annotation table including genomic positions and simulation-specific metadata.

```r
data("sim_ex")

map <- sim_ex$map
```

To use the genotype data throughout the workflow, we first create a temporary GDS file.

```r
gds_path <- tempfile(fileext = ".gds")
gds <- create_gds_from_geno(geno = sim_ex$GTs, map, gds_path)
```

0.3 Estimating LD decay

The first step is to estimate chromosome-wise LD decay. This is done in two stages.

We first run a preliminary analysis on a subset of markers to obtain a rough estimate of the sliding window size needed for downstream LD calculations.

```
ld_decay_pre <- compute_LD_decay(
  gds,
  n_win_decay = 5,
  max_SNPs_decay = 5000,
  slide = 1000,
  cores = cores
)
```

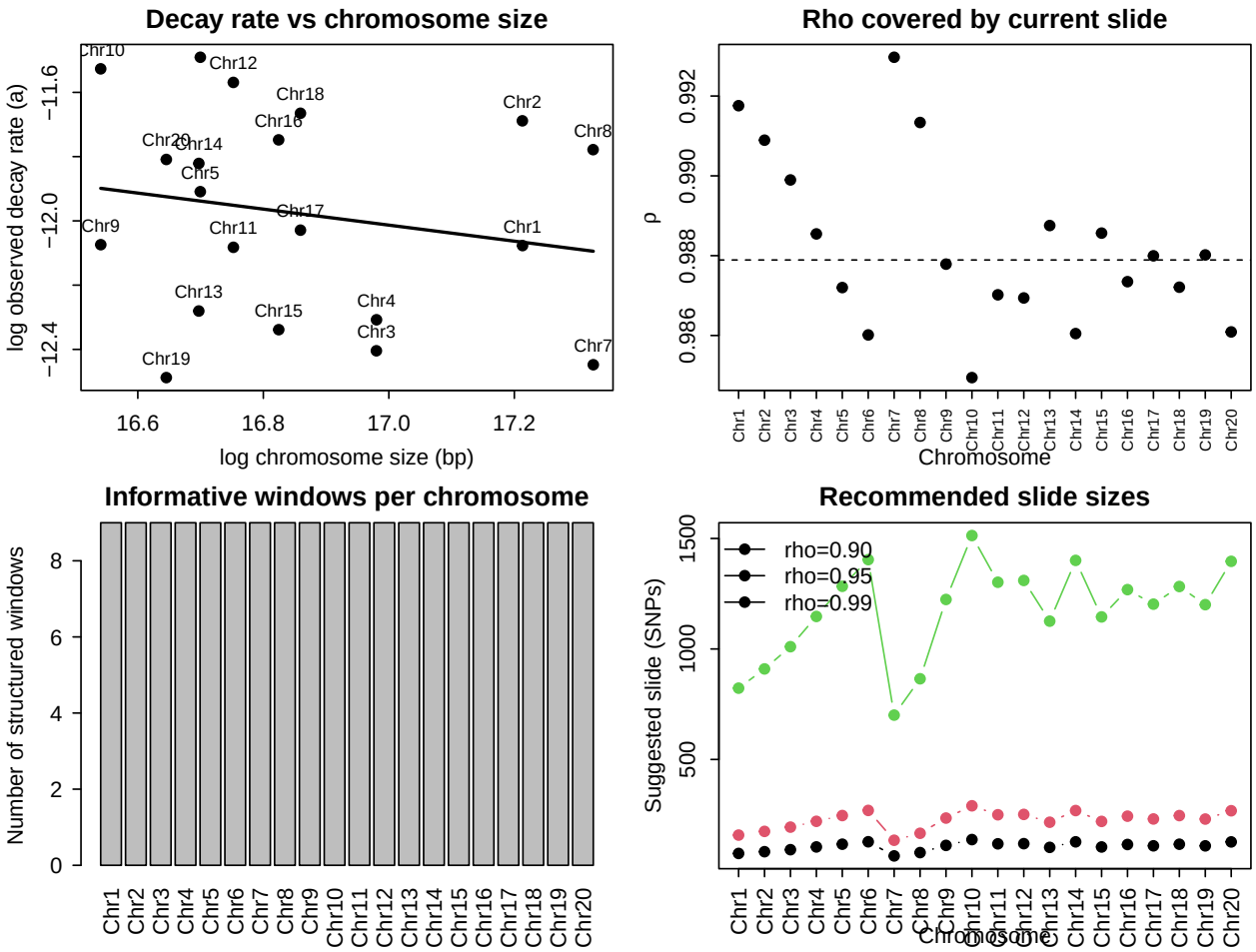
This gives a summary of LD-decay based on the subset of the data.

```
ld_decay_pre
#> <ld_decay>
#>
#> Run parameters:
#>   slide window: 1,000 SNPs
#>   background LD quantile q: 0.95
#>   chromosomes analysed: 20
#>   keep_el: FALSE
#>
#> Decay parameter summary:
#>   predicted a:median =6.39e-06 range = [5.59e-06, 6.8e-06]
#>   predicted c: 1
#>   structured decay windows:median =9 range = [9, 9]
#>
#> Current slide window coverage:
#>   rho covered:median =0.988 range = [0.985, 0.993]
#>   (slide = 1,000 SNPs)
#>
#> Suggested slide windows for target rho:
#>   rho = 0.90 : median = 110 SNPs , p90 = 127 SNPs , max = 138 SNPs
#>   rho = 0.95 : median = 233 SNPs , p90 = 269 SNPs , max = 290 SNPs
#>   rho = 0.99 : median = 1,213 SNPs , p90 = 1,401 SNPs , max = 1,513 SNPs
#>   (median = typical chromosome, p90 = covers most chromosomes)
#>
#> Stored components:
#>   by_chr, decay_sum, decay_model, recommendation, params
```

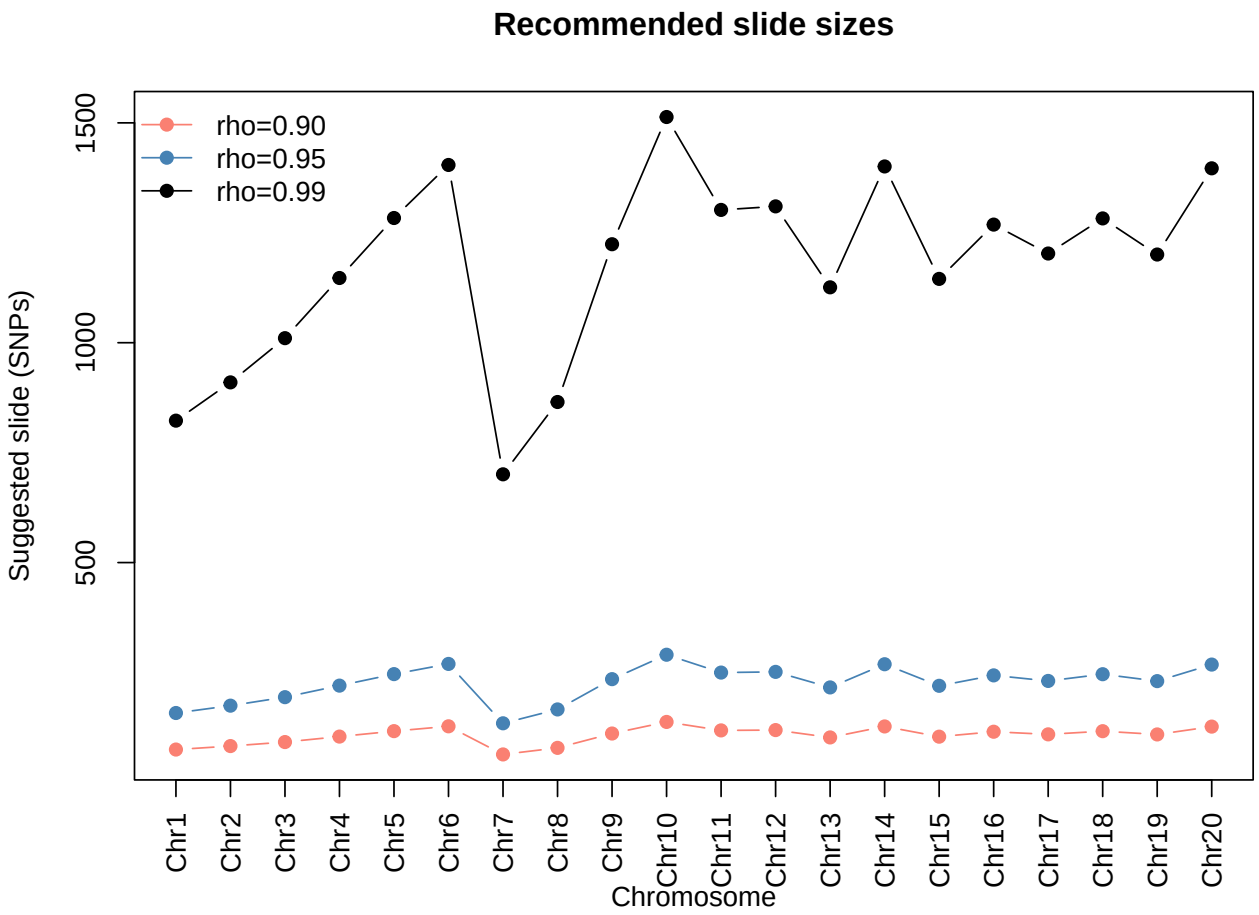
In this simulated example, chromosomes are relatively short, so only five decay windows per chromosome are used. For empirical data sets, the default settings are usually more appropriate.

The preliminary LD-decay fit can be used to select a more suitable sliding window for the full analysis. We can visualize LD-decay, and use the recommended new slide window for estimating LD.

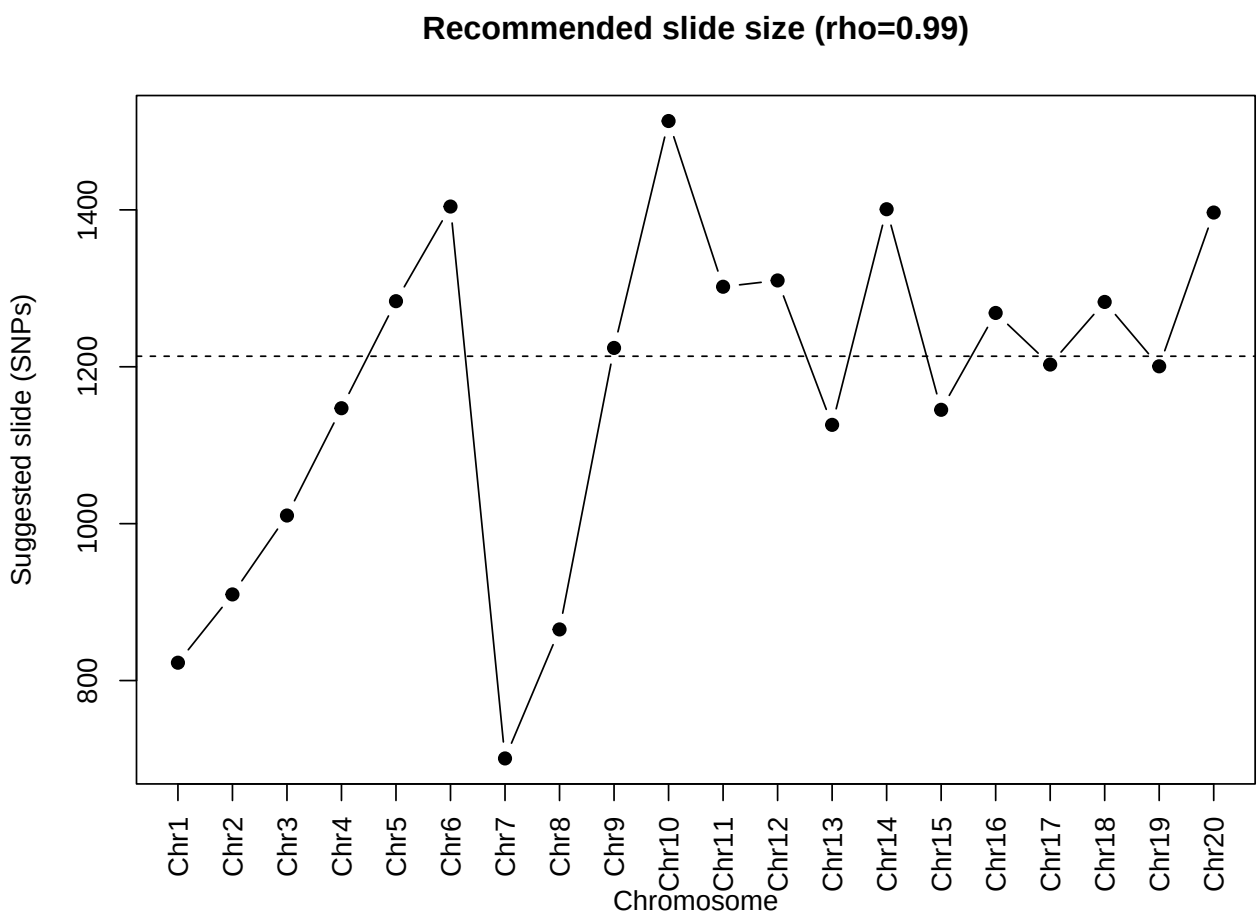
```
plot(ld_decay_pre, type = "summary")
```



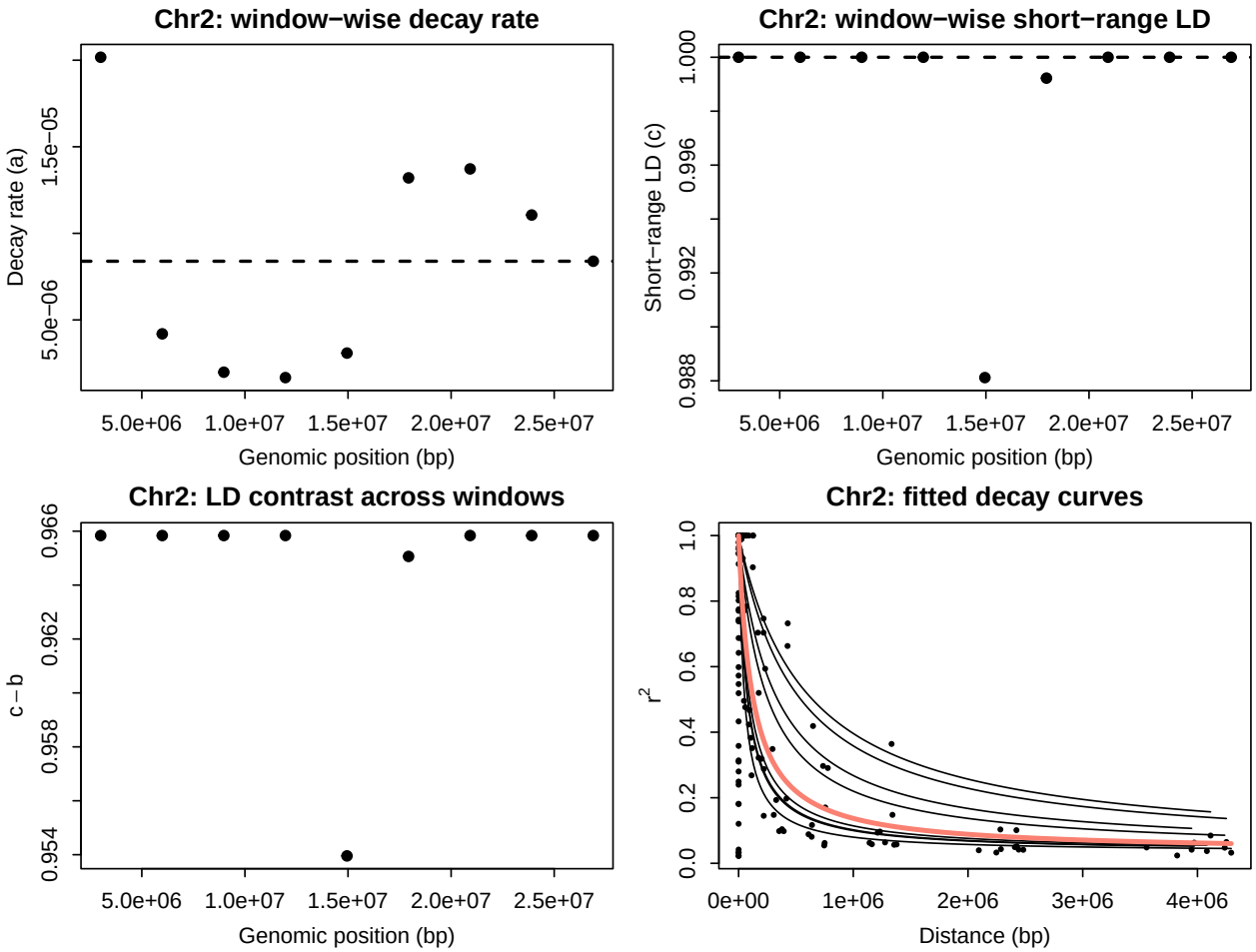

```
plot(ld_decay_pre, type = "recommendation")
```




```
plot(ld_decay_pre, type = "recommendation", rho = 0.99)
```




```r
plot(ld_decay_pre, type = "chr", chr = "Chr2")
```



Here we use the mean recommended SNP window corresponding to `rho = 0.99`.

```
new_slide_window <- mean(ld_decay_pre$decay_sum$slide_snp_rho_0.99)

ld_decay <- compute_LD_decay(
  gds,
  n_win_decay = 5,
  max_SNPs_decay = Inf,
  keep_el = TRUE,
  slide = new_slide_window,
  cores = cores
)
```

The second run uses all SNPs and stores LD edge lists for downstream analyses. Note that the edge list of pairwise r^2 values can be saved to files (using `fwrite` and `fread`) instead to save RAM by specifying e.g. `el_data_folder = "./el/"`.

0.4 Repeated outlier-region draws

Next, we generate repeated outlier-region draws across a grid of LD-support thresholds (`rho_w`) and a range of outlier-region detection parameters.

In this example, we use two precomputed test statistics from the simulated data: `lfmm_F` and `emx_F`. We also pre-compute the ld_w vectors for a range of rho values.

```
ld_ws <- precalculate_ld_w(rho=c(seq(0.75,0.95,by=0.05),0.99),ld_decay)
#>   |                                                                      |
#ld_decay$by_chr$Chr1$el

draws <- ld_rho_draws(gds,
                      ld_decay  = ld_decay,
                      F_vals     = map[,.(lfmm_F,emx_F)],
                      q_vals     = NULL,
                      n_draws    = 100,
                      stat_type  = "q",
                      rho        = NULL,
                      ld_ws      = ld_ws,
                      rho_d_lim  = list(min=0.9,max=0.99),
                      rho_ld_lim = list(min=0.9,max=0.99),
                      alpha_lim  = list(min=0.3,max=2),
                      lmin_lim   = list(min=1,max=10),
                      cores      = cores,
                      mode       = "joint"
)
#>   |                                                                      |
```

Here, `alpha_lim` is defined on the `-log10` scale. For example, the lower bound `0.6` corresponds to a q-value threshold of approximately `1 / 10^0.6 = 0.25`.

0.5 Manhattan-style visualization

A convenient way to summarize the results is to compute SNP-level consistency scores and plot them together with the original scan statistics.

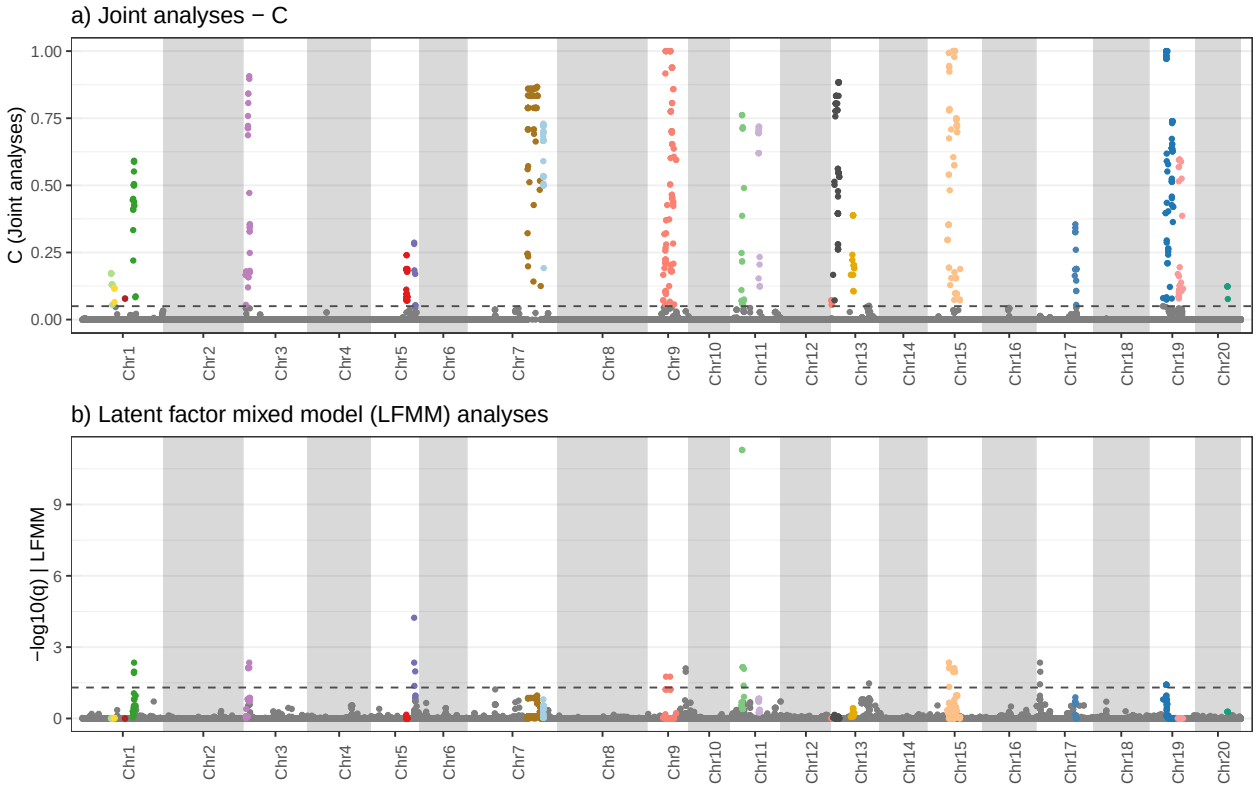
We first add a transformed q-value for LFMM:

```
map[, log_lfmm_q := -log10(lfmm_q)]
```

We can then plot the joint consistency score together with the LFMM q-values, coloring points by detected outlier region.

```
plot_manhattan(
  map,
  gds,
  ld_decay,
  draws,
  sign_th = 0.05,
  mode = "joint",
  sign_if = "greater",
  rho_d = 0.9,
  rho_ld = 0.9,
  y_vars = c("Joint_C", "log_lfmm_q"),
  y_labels = c("C (Joint analyses)", "-log10(q) | LFMM"),
  titles = c(
    "a) Joint analyses - C",
    "b) Latent factor mixed model (LFMM) analyses"
  ),
  thresholds = c(0.05, -log10(0.05)),
  col_var = "OR_id",
  col_vector = NULL
```

```
)
```



In this plot, the upper panel shows the SNP-level consistency score across repeated outlier-region draws, while the lower panel shows the original LFMM result. The outlier regions are always based on the first in the vector of method names for y_vars, in this case "Joint_C".

0.6 Benchmarking with simulated truth

Because this is simulated data, we can compare detected outlier regions to the known QTNs.

We first define which focal QTNs count as true positives. Following the classification used in the original analyses, a SNP is considered potentially detectable if it is both sufficiently close to and sufficiently correlated with a focal QTN.

```
ld_test_th <- 0.75

d_test_th <- 0.95

p_Va_th <- 0.05
```

To express LD and distance relative to chromosome-specific LD decay, we map the decay parameters back onto the SNP table.

```
as <- setNames(ld_decay$decay_sum$a, ld_decay$decay_sum$Chr)
bs <- setNames(ld_decay$decay_sum$b, ld_decay$decay_sum$Chr)
cs <- setNames(ld_decay$decay_sum$c, ld_decay$decay_sum$Chr)

a <- as[map$Chr]
b <- bs[map$Chr]
C <- cs[map$Chr]
```

```
map[, rho_ld := 1 - (max_LD_with_QTN - b) / (C - b)]
map[, rho_d := a * bp_to_focal_QTN / (a * bp_to_focal_QTN + 1)]
map[type == "QTN", rho_ld := 1]
```

We then retain only SNPs that satisfy the LD- and distance-based criteria relative to at least one focal QTN.

```
map_filt <- map[rho_ld > ld_test_th & rho_d < d_test_th]
true_QTN_focal <- unique(map_filt[p_Va > p_Va_th, focal_QTN])
```

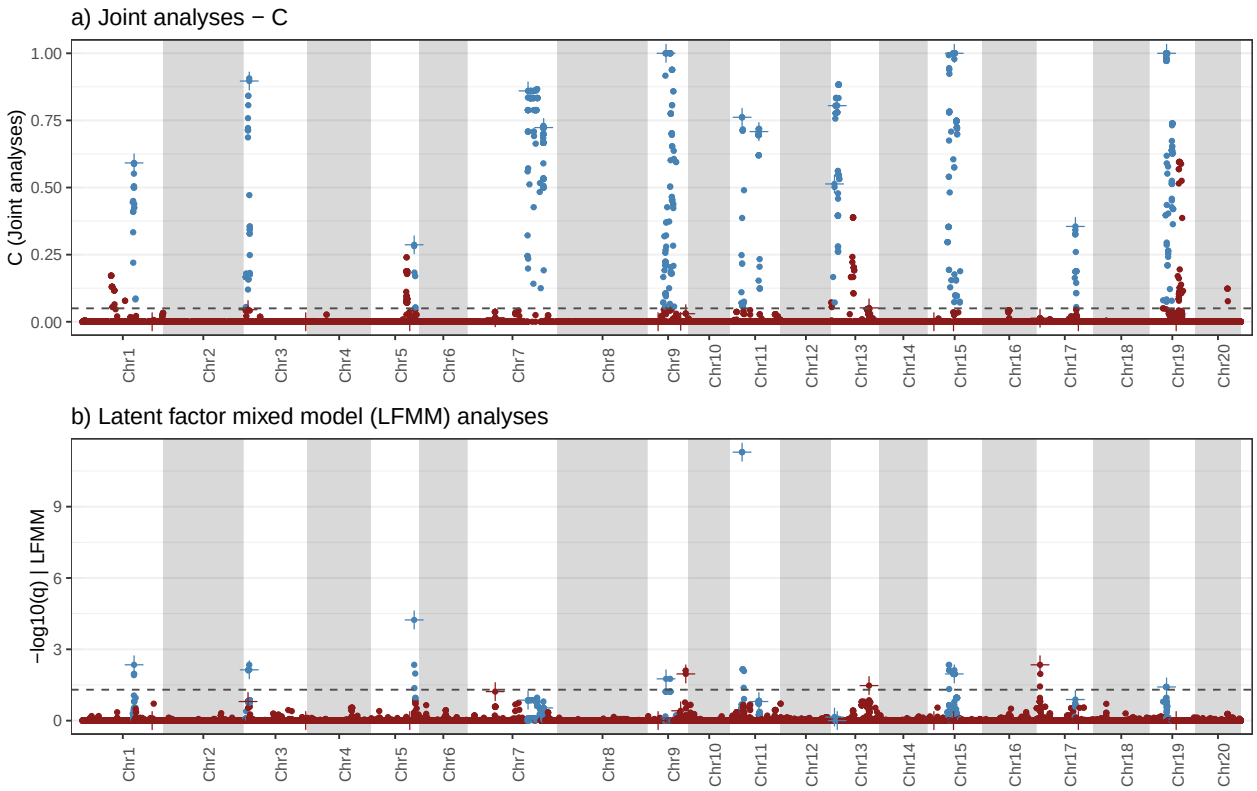
Next, we compute consistency scores, add outlier-region labels, and classify detected regions as true or false positives.

```
map_manh <- add_consistency_to_map(
  map,
  consistency_obj = consistency_score(draws$draws)
)

map_manh <- add_ORs(
  gds,
  ld_decay,
  map_manh,
  stat = "Joint_C",
  sign_th = 0.05,
  sign_if = "greater",
  mode = "joint",
  rho_d = 0.9,
  rho_ld = 0.9,
  l_min = 1
)

ORs <- split(
  map_manh[OR_id != "ns"]$marker,
  map_manh[OR_id != "ns"]$OR_id
)

OR_focals <- vapply(ORs, function(or) {
  sub <- map_filt[marker %in% or]
  if (nrow(sub) == 0L) return(NA_character_)
  sub[which.max(rho_ld), focal_QTN]
}, character(1))

OR_focals <- unique(na.omit(OR_focals))
true_pos_focals <- OR_focals[OR_focals %in% true_QTN_focal]

true_ORs <- sapply(true_pos_focals, function(t_focal) {
  map_manh[focal_QTN == t_focal][order(-max_LD_with_QTN)][1, OR_id]
})

true_ORs <- true_ORs[true_ORs != "ns"]

map_manh[, OR_test := ifelse(OR_id %in% true_ORs, "True pos", "False pos")]
```

The resulting Manhattan-style plot highlights true and false positive outlier regions. Because the threshold settings for OR clustering were low (0.9), we can see two false positives on chromosomes 13 and 19. This illustrates "overclustering" and if you repat the above with `rho_d = rho_ld = 0.99`, you will see that they

will be considered part of the adjacent true positive ORs. This illustrates the effect of the clustering thresholds on the numbers of true and false positive ORs (TP and FP, respectively). Note that on the even numbered neutal chormosomes, no outlier regions are found, regardless of clustering thresholds.

```
layout <- prep_manhattan(
  map_manh[, .(
    Pos,
    Chr,
    marker,
    OR_test,
    log_lfmm_q,
    Joint_C,
    type
  )]
)

plot_manhattan_gg(
  layout,
  y_vars = c("Joint_C", "log_lfmm_q"),
  y_labels = c("C (Joint analyses)", "-log10(q) | LFMM"),
  titles = c(
    "a) Joint analyses - C",
    "b) Latent factor mixed model (LFMM) analyses"
  ),
  thresholds = c(0.05, -log10(0.05)),
  col_var = "OR_test",
  shape_var = "type",
  ncol = 1,
  col_vector = c("firebrick4", "steelblue")
)
```



0.7 Area under the PR curve

To summarize performance across repeated draws, we can compute precision-recall summaries and the normalized area under the cumulative PR curve.

For the q-based workflow:

```
PR_dat <- cbind(
  method = "joint",
  get_PR(draw_ORs = draws$draws$OR,
         map = map,
         map_filt = map_filt,
         p_Va_th = 0.05,
         cores = cores)
)

get_AUC_OR(PR_dat)$AUC$AUC_norm
#> [1] 0.5691627
```

We can also repeat the analysis using the consistency score itself as the input statistic.

First, we construct a table of SNP-level consistency scores:

```
C_scores <- add_consistency_to_map(
  map,
  consistency_score(draws$draws)
)[, .(marker = marker, Joint_C = Joint_C)]
```

We then generate repeated outlier-region draws directly from the consistency score.

```r
draws_C <- ld_rho_draws(
  gds,
  ld_decay = ld_decay,
  F_vals = NULL,
  C_scores = C_scores[, .(Joint_C)],
  q_vals = NULL,
  n_draws = 1000,
  stat_type = "C",
  rho_d_lim = list(min = 0.9, max = 0.99),
  rho_ld_lim = list(min = 0.9, max = 0.99),
  alpha_lim = NULL,
  lmin_lim = NULL,
  C_lim = list(min = 0, max = 0.5),
  cores = cores,
  mode = "joint"
)
#>   |                                                                              |
```

Finally, we compute the corresponding PR summary and normalized AUC.

```r
PR_C <- cbind(
  method = "joint",
  get_PR(draw_ORs = draws_C$draws$OR, map, map_filt = map_filt, p_Va_th = 0.05, cores = cores)
)

get_AUC_OR(PR_C)$AUC$AUC_norm
#> [1] 0.5070434
```

0.8 Summary

This vignette illustrated a complete `LDscnR` workflow using simulated data. Starting from genotype and SNP metadata, we:

- estimated chromosome-wise LD decay,
- generated repeated LD-aware outlier-region draws,
- summarized robustness using SNP-level consistency scores,
- visualized results in Manhattan-style plots,
- evaluated performance against known simulated QTNs.

For empirical data, the same general workflow applies, minus the simulation-specific benchmarking steps.